

5 El procesador segmentado

La segmentación (*pipelining*) es una técnica por medio de la cual la arquitectura de un computador se divide múltiples etapas, cada una de las cuales, se corresponde con una de las fases experimentadas por una instrucción desde que es leída de memoria hasta que sus resultados son escritos en el banco de registros. Este enfoque permite que distintas instrucciones ocupen distintas etapas de manera concurrente (incrementando así el paralelismo a nivel de instrucción), y requiere introducir en la arquitectura registros de etapa con los que almacenar de manera temporal el contexto de una instrucción al final de cada etapa, es decir, el conjunto de datos y señales de control asociados a la instrucción, los cuales serán transferidos a la etapa inmediatamente posterior en el siguiente ciclo de reloj.

Puede verse en la Figura 5.1 un diagrama en el que se compara el flujo de ejecución de un computador monociclo frente al de uno segmentado.

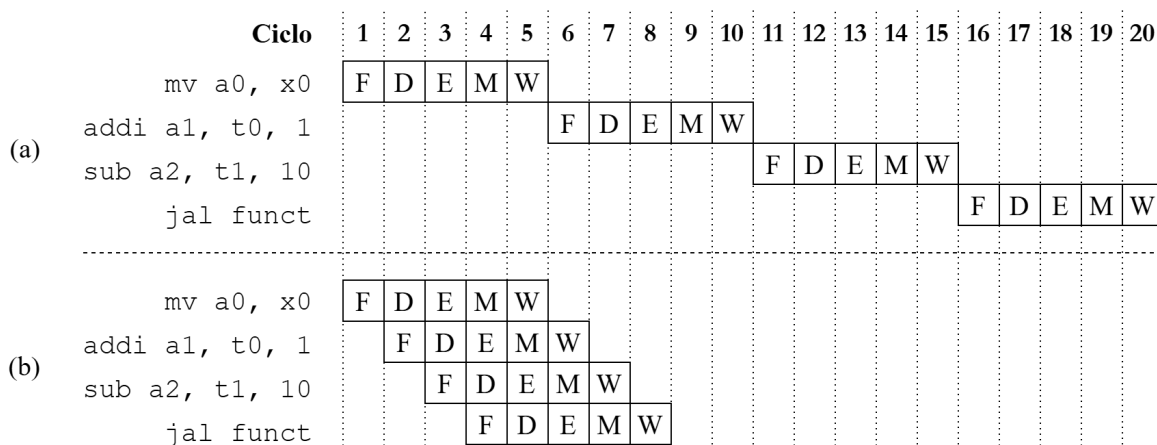


Figura 5.1: Ejecución de un programa en un computador con diseño monociclo (a) y segmentado (b)

Esta técnica conlleva una mayor complejidad arquitectónica y el surgimiento de una serie de riesgos o *hazards* generados bien por las dependencias de datos entre las instrucciones de un programa (ver Figuras 5.2 y 5.3), o bien por la ejecución de instrucciones de control que pueden o no alterar el flujo de ejecución de este (ver Figura 5.4). En función de las decisiones de diseño que se tomen para abordar la problemática que representan estos fenómenos, su aparición desembocará en la introducción de detenciones (*stalls*) en el flujo de ejecución, o la necesidad de incluir en el diseño mecanismos adicionales como el *forwarding*, consistente en el conexionado de determinadas señales de datos o de control hacia etapas anteriores en el pipeline.

En la Figura 5.2 aparecen identificados los distintos tipos de dependencias de datos que pueden darse durante la ejecución de un programa. La dependencia RAW (a) se da cuando uno de los operandos de una instrucción es un registro (`x1`, señalado en color morado) cuyo valor deberá ser previamente modificado

<code>add x1, x2, x3</code> <code>sub x4, x1, x5</code> (a)	<code>add x1, x2, x3</code> <code>add x1, x4, x5</code> (b)	<code>add x4, x1, x2</code> <code>add x1, x3, x5</code> (c)
---	---	---

Figura 5.2: Ejemplos de riesgos de datos. (a) RAW (*Read After Write*): lectura tras escritura. (b) WAW (*Write After Write*): escritura tras escritura. (c) WAR (*Write After Read*): escritura tras lectura

por otra instrucción separada de la primera 3 ciclos o menos. Esto es así ya que la primera instrucción, la que lee el dato, deberá poder observar ese dato actualizado de modo que pueda elaborar un resultado coherente con el orden del programa. Es decir que, en un principio, la segunda instrucción no debería poder leer sus operandos (en su *decode*) hasta la escritura (*writeback*) del resultado calculado por la primera. Se ejemplifica esta situación por medio del diagrama presentado en la Figura 5.3.

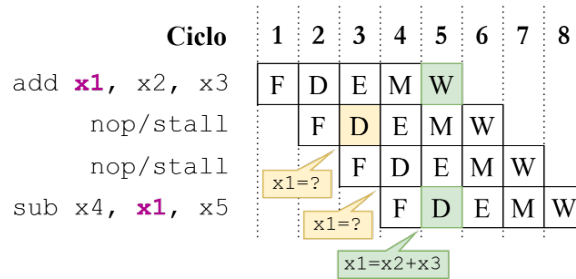


Figura 5.3: Flujo de ejecución de dos instrucciones implicadas en una dependencia RAW. Se señala en color verde el instante a partir del cual la segunda instrucción podrá leer el dato actualizado en el registro `x1` (ciclo 5).

Los otros dos tipos de dependencias presentados, señalados con las letras *b* y *c*, no representan un riesgo real ya que dos instrucciones pueden leer o escribir de manera consecutiva sobre el mismo registro sin que ello suponga la pérdida de coherencia en el dato, al igual que sucede con las escrituras sobre un registro que fue previamente leído.

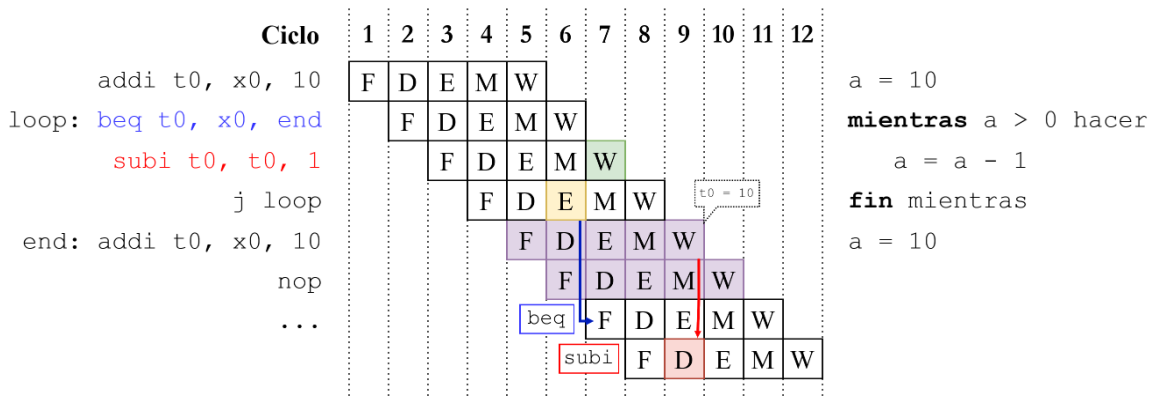


Figura 5.4: Representación del flujo de ejecución de un programa sobre una arquitectura segmentada sin mecanismos para resolver riesgos de control.

Sobre el ejemplo de riesgo de control presentado en la Figura 5.4, el resultado de la instrucción de resta (`subi`) se escribe (writeback señalado en color verde) en el banco de registros antes incluso de evaluar la condición en la instrucción de salto, dejando el valor del registro `t0` en un estado inconsistente. Además, dado que el destino del salto de la instrucción `jal` no se calculará hasta su ejecución, habrá dos ciclos en los que se introducirán las dos instrucciones inmediatamente posteriores (señaladas en color morado), una de las cuales reinicia el valor de la variable `a` haciendo que el bucle no tenga final. El resultado de la instrucción `subi` en la primera iteración se sobrescribe con el del segundo `addi`.

Por último, y a pesar de las limitaciones comentadas, se debe destacar que la segmentación de instrucciones permite obtener un speedup teórico igual al número de etapas en que se haya decidido segmentar la arquitectura, teniendo siempre en cuenta que aspectos como la presencia y resolución de los riesgos mencionados, la latencia en los accesos a memoria o el tiempo desigual entre los caminos críticos de las diferentes etapas, impedirán alcanzar ese rendimiento máximo ideal.

5.1. Arquitectura de un registro de etapa

La función de un registro de etapa es almacenar de manera transitoria el conjunto de señales de control y datos emitidos al término de una etapa de modo que, en el siguiente ciclo de reloj, esas mismas señales puedan ser transferidas a la etapa inmediatamente posterior en el pipeline. De este modo se consigue desacoplar temporalmente la ejecución de las distintas etapas permitiendo que varias instrucciones ocupen el pipeline de manera concurrente.

Desde el punto de vista de la implementación en Chisel, un registro de etapa como el mostrado en la Figura 5.5 se compone, en primer lugar, de un bundle en el que se definen las entradas y salidas de este. Estas se corresponderán con el conjunto de señales de control y datos que deban avanzar de una etapa a la siguiente. Posteriormente, cada entrada de datos/control se conectará a la entrada de un registro específico de esa señal y, la salida del registro, a su correspondiente salida en el bundle.

```

1  class FD extends Module {
2
3    val io = IO(new Bundle {
4      val f_instruction      = Input(UInt(Parameters.InstructionWidth))
5      val f_instruction_address = Input(UInt(Parameters.AddrWidth))
6
7      val d_instruction      = Output(UInt(Parameters.InstructionWidth))
8      val d_instruction_address = Output(UInt(Parameters.AddrWidth))
9    })
10
11    val instruction      = RegInit(0.U(Parameters.InstructionWidth))
12    val instruction_address = RegInit(0.U(Parameters.AddrWidth))
13
14    instruction      := io.f_instruction
15    instruction_address := io.f_instruction_address
16
17    io.d_instruction      := instruction
18    io.d_instruction_address := instruction_address
19  }

```

Figura 5.5: Implementación básica del registro de etapa que separa las fases de *fetch* y decodificación.

5.2. Segmentando el núcleo en dos etapas

El primer paso hacia la segmentación del núcleo en las cinco etapas marcadas como objetivo del trabajo consistió en segmentarlo primeramente en dos, aislando la fase de *fetch* respecto de las unidades funcionales implicadas en el resto de etapas.

Esta división no conlleva la aparición de riesgos de datos puesto que tanto la escritura como la lectura de estos forman parte de la misma fase dentro del ciclo de procesamiento de las instrucciones. Sin embargo, esta segmentación sí da pie a que, durante la ejecución de un programa, puedan aparecer riesgos de control tal y como se muestra en el ejemplo presentado en la Figura 5.6, donde la ausencia de mecanismos que logren mitigar el riesgo de control provoca la lectura de la instrucción señalada en color rojo antes de que se haya evaluado siquiera la condición en la instrucción de salto, ejecutando así la instrucción de resta y depositando en el registro `x1` un valor incoherente con la lógica del programa.

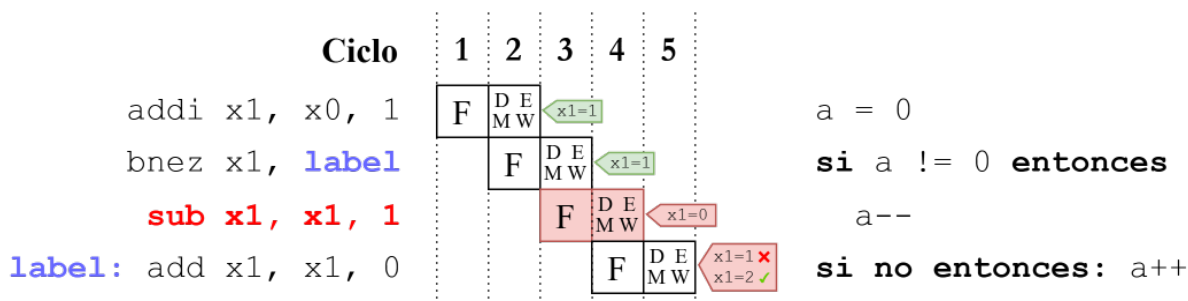


Figura 5.6: Representación del flujo de ejecución de un programa sobre una arquitectura segmentada en dos etapas sin mecanismos para resolver riesgos de control.

Para verificar la existencia o no del riesgo de control tras la introducción en la arquitectura del primer registro de segmentación se desarrolló un sencillo programa en lenguaje ensamblador el cual incorpora instrucciones de salto (condicional, incondicional y llamada a función). Este programa (cuyo código se muestra en el Cuadro 5.1) fue compilado, cargado en memoria y ejecutado en el computador simulado por medio del test de Chisel mostrado en la Figura 5.7.

El proceso de compilación se llevó a cabo mediante un *Makefile* que compila los ficheros de código fuente en lenguaje C y genera ejecutables en formato ELF. Posteriormente, dichos ejecutables se convierten a binario plano, que es el formato esperado por el cargador del computador. A continuación, ya sea manualmente o mediante un *target* específico del *Makefile*, el fichero binario generado se copia a la ruta `src/main/resources` del proyecto. De este modo, al construir el proyecto con `sbt`, los binarios presentes en dicho directorio se incluyen automáticamente en el *classpath* tanto de la aplicación como de los tests.

TODO: *quizás podría meter un diagrama de flujo con las fases del proceso de compilación y carga de código.*

En cuanto al test implementado, este consta, en primer lugar, de un bucle de inicialización en el que se espera a que el cargador lea el contenido del binario y lo deposite en la memoria del computador. Posteriormente, se ejecuta un segundo bucle en el que se hace avanzar el reloj 50 ciclos. En cada iteración se realiza, mediante el método `poke`, una lectura de depuración sobre la memoria. De este modo se fuerza al simulador a no introducir optimizaciones en los bucles con las que se impediría observar el flujo de ejecución del programa a lo largo de eso 50 ciclos de reloj.

El primero de los problemas que se observó al examinar la traza generada por el test fue la falta de sincronismo entre el valor de la dirección de la instrucción respecto de la propia instrucción presente en la misma etapa, tal y como aparece señalado en la Figura 5.8. La consecuencia que esto tiene sobre el flujo de ejecución de los programas es que las instrucciones de salto no pueden realizar correctamente el cómputo de la dirección efectiva del salto. Tal y como se observa en la traza de la Figura 5.8, al comienzo del ciclo marcado en la posición en que se encuentra el indicador vertical (43 ps) la instrucción

jal suma_uno (0x010000EF), cargada en la señal instr@DEMW, observa como operandos para el cálculo del destino del salto los valores 0x10 y 0x1014 que son, respectivamente, el valor del inmediato cargado en la señal ex_io_immediate, y el que debería ser el valor de la dirección de la instrucción de salto. No obstante, esta última señal corresponde al valor de la dirección de la instrucción inmediatamente posterior al salto.

```

1 class SegRegFDTest extends AnyFlatSpec with ChiselScalatestTester {
2   behavior.of("Pipelined CPU")
3   it should "run a simple program with a segmentation register placed in between fetch and
4     decode phases" in {
5     test(new TestTopModule("simple_subroutine.asmbin"))
6       .withAnnotations(TestAnnotations.annos) { c =>
7       while(!c.io.instruction_valid.peek().litToBoolean) {
8         c.clock.step()
9         c.io.mem_debug_read_address.poke(4.U)
10      }
11
12      for (i <- 1 to 50) {
13        c.clock.step()
14        c.io.mem_debug_read_address.poke((i * 4).U)
15      }
16
17      c.io.regs_debug_read_address.poke(6.U) // x6 => t1
18      c.io.regs_debug_read_data.expect(5.U)
19    }
20  }

```

Figura 5.7: Código del test con el que se pone a prueba el funcionamiento de un programa sencillo sobre el núcleo segmentado en dos fases.

Código máquina	Ensamblador	Pseudocódigo
00000000 <_start>: 0: 00000313 4: 00400393 00000008 <loop_start>: 8: 0063ca63 c: 00030513 10: 010000ef 14: 00050313 18: ff1ff06f 0000001c <loop_end>: 1c: 0000006f 00000020 <suma_uno>: 20: 00050293 24: 00128293 28: 00028513 2c: 00008067	li t1,0 li t2,4 blt t2,t1,loop_end mv a0,t1 jal suma_uno mv t1,a0 j loop_start j loop_end mv t0,a0 addi t0,t0,1 mv a0,t0 ret	t1 = 0 t2 = 4 mientras t1 < t2 t1 = suma_uno(t1) función suma_uno(x) retornar x + 1

Cuadro 5.1: Desensamblado, código ensamblador y pseudocódigo del programa empleado en la depuración de la arquitectura.

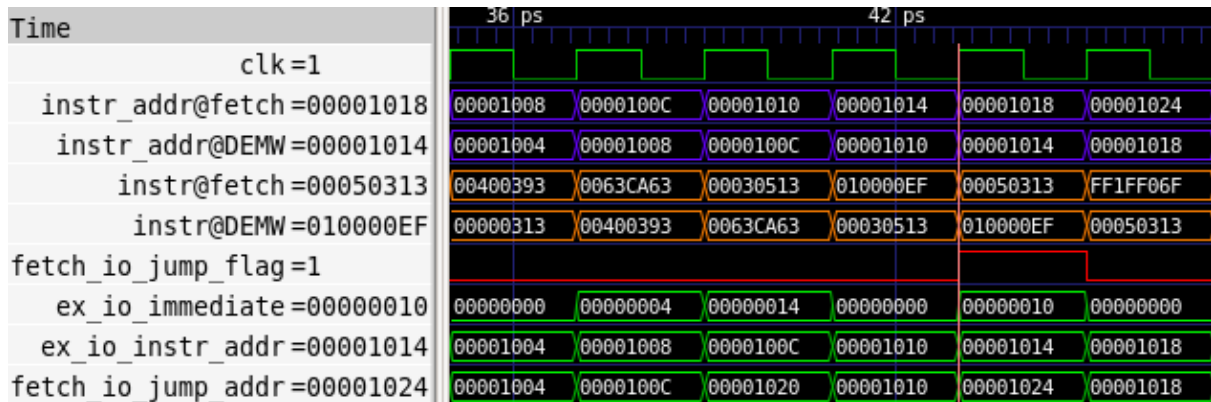


Figura 5.8: Sección de la traza obtenida tras la ejecución del test mostrado en la Figura 5.7

El diseño original emplea como memoria una instancia de la clase *SyncReadMem*. Este tipo de memoria no permite realizar lecturas de manera asíncrona/combinacional, lo cual implica que, cuando se solicita una lectura, el dato leído no sale de la memoria hasta el siguiente flanco ascendente de reloj, impidiendo que se mantenga la sincronía entre las instrucciones en cada fase y la dirección de la instrucción observada en esa misma fase.

Para solucionarlo se cambia, dentro de la clase *Memory*, el tipo de memoria a una de la clase *Mem*, de forma que la lectura del dato se lleve a cabo en el instante mismo en que se solicita. De este modo, tal y como puede observarse en la traza presentada en la Figura 5.9, el valor de la dirección de la instrucción en cada fase se corresponde con la de la instrucción que se está procesando esa misma fase, de tal modo que ahora el cálculo del *target* en la instrucción de salto sea correcto (*suma_uno@0x1020*).

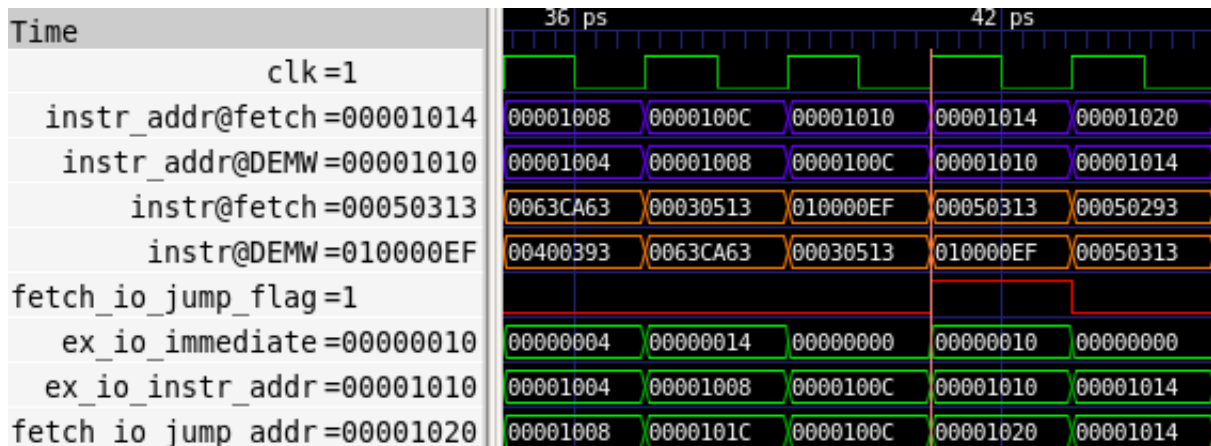


Figura 5.9: Misma sección de la traza pero con una instancia de la clase *Mem* como entidad de memoria en lugar de *SyncReadMem*.

A pesar de esta corrección, puede observarse que el flujo de ejecución del programa sigue siendo erróneo ya que, siguiendo a la instrucción de salto (tras la marca vertical), en la segunda fase logra introducirse y ejecutarse la instrucción inmediatamente posterior, la cual es un *mv* (0x0014: *mv t1, a0*). En el caso concreto de la primera iteración del bucle, la ejecución de esta instrucción es inócua ya que en ese punto los registros *t1* y *a0* albergan el mismo valor. No obstante a esto, la inserción de la instrucción siguiente al salto toma una relevancia significativa cuando se trata de otra instrucción de salto, tal y como ocurre tras el salto incondicional al final del bucle. La ejecución de este segundo salto, el cual forma

parte del bucle de final de programa, provoca que este llegue a término tras la primera iteración del bucle (véase Figura 5.9).

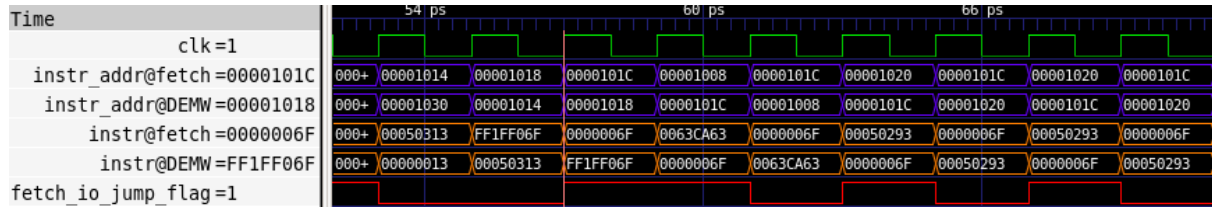


Figura 5.10: Sección de la traza en la que se muestra la pronta ejecución del bucle final.

La solución final para resolver esta problemática consiste en la introducción de un par de modificaciones en la fase de fetch para hacer que la arquitectura introduzca una operación **nop** tras cada instrucción de salto en cuya segunda fase se determine que este deba ser tomado. Estas modificaciones se realizan sobre la lógica que hace avanzar el contador de programa.

En la nueva implementación, tal y como se observa en los códigos presentados en el Cuadro 5.2, se introduce un **nop** en el pipeline siempre y cuando se cumplan las dos condiciones siguientes: en primer lugar, que el cargador no haya terminado su trabajo y, en segundo lugar, que en la segunda fase de una instrucción de salto se haya determinado que este deba ser tomado (originalmente solo se contemplaba la primera condición).

Implementación original

```
class InstructionFetch extends Module {  
  
  val io = IO(new Bundle {  
    val jump_flag_id      = Input(Bool())  
    val jump_address_id   = Input(UInt(Parameters.AddrWidth))  
    val instruction_read_data = Input(UInt(Parameters.DataWidth))  
    val instruction_valid  = Input(Bool())  
  
    val instruction_address = Output(UInt(Parameters.AddrWidth))  
    val instruction         = Output(UInt(Parameters.InstructionWidth))  
  })  
  
  val pc = RegInit(ProgramCounter.EntryAddress)  
  
  when(io.instruction_valid) {  
    pc      := Mux(io.jump_flag_id, io.jump_address_id, pc + 4.U)  
    io.instruction := io.instruction_read_data  
  }.otherwise {  
    pc      := pc  
    io.instruction := 0x00000013.U  
  }  
  
  io.instruction_address := pc  
}
```

Código adaptado

```
class InstructionFetch extends Module {  
  
  val io = IO(new Bundle {  
    val jump_flag_id      = Input(Bool())  
    val jump_address_id   = Input(UInt(Parameters.AddrWidth))  
    val instruction_read_data = Input(UInt(Parameters.DataWidth))  
    val instruction_valid  = Input(Bool())  
  
    val instruction_address = Output(UInt(Parameters.AddrWidth))  
    val instruction         = Output(UInt(Parameters.InstructionWidth))  
  })  
  
  val pc = RegInit(ProgramCounter.EntryAddress)  
  
  when(io.instruction_valid && !io.jump_flag_id) {  
    pc      := pc + 4.U  
    io.instruction := io.instruction_read_data  
  }.otherwise {  
    pc      := io.jump_address_id  
    io.instruction := 0x00000013.U  
  }  
  
  io.instruction_address := pc  
}
```

Cuadro 5.2: Comparación entre la implementación original y la implementación adaptada del módulo InstructionFetch.